

get_scheduler

https://huggingface.co/docs/transformers/main/en/main_classes/optimizer_scheduler

Detailed Documentation

`class`

`transformers.Adafactor`

class

```
transformers.Adafactor: https://github.com/huggingface/transformers/blob/main/src/transformers/optimization.py#L100

    "params", "val": "", {"name": "lr", "val": " = None"}, {"name": "eps", "val": " = (1e-30, 0.001)"}, {"name": "clip_threshold", "val": " = 1.0"}, {"name": "decay_rate", "val": " = -0.8"}, {"name": "beta1", "val": " = None"}, {"name": "weight_decay", "val": " = 0.0"}, {"name": "scale_parameter", "val": " = True"}, {"name": "relative_step", "val": " = True"}, {"name": "warmup_init", "val": " = False"}] - **params** ( Iterable[nn.parameter.Parameter] ) --
```

Iterable of parameters to optimize or dictionaries defining parameter groups.

- `lr` (`float`, *optional*) --

The external learning rate.

- `eps` (`tuple[float, float]`, *optional*, defaults to `(1e-30, 0.001)`) --


```
None"}, {"name": "start", "val": " = 1"}]
```

Scheduler names for the parameter `lr_scheduler_type` in `[TrainingArguments]`
(/docs/transformers/main/en/main_classes/trainer#transformers.TrainingArgument

By default, it uses "linear". Internally, this retrieves `get_linear_schedule_with_warmup` scheduler from `[Trainer]`
(/docs/transformers/main/en/main_classes/trainer#transformers.Trainer).

Scheduler types:

- "linear" = [`get_linear_schedule_with_warmup`] (/docs/transformers/main/en/main_classes/optimizer_schedules#transformers.optimization.schedules.LinearScheduleWithWarmup)
- "cosine" = [`get_cosine_schedule_with_warmup`] (/docs/transformers/main/en/main_classes/optimizer_schedules#transformers.optimization.schedules.CosineScheduleWithWarmup)
- "cosine_with_restarts" = [`get_cosine_with_hard_restarts_schedule_with_warmup`] (/docs/transformers/main/en/main_classes/optimizer_schedules#transformers.optimization.schedules.CosineScheduleWithHardRestartsWithWarmup)
- "polynomial" = [`get_polynomial_decay_schedule_with_warmup`] (/docs/transformers/main/en/main_classes/optimizer_schedules#transformers.optimization.schedules.PolynomialDecayScheduleWithWarmup)
- "constant" = [`get_constant_schedule`] (/docs/transformers/main/en/main_classes/optimizer_schedules#transformers.optimization.schedules.ConstantSchedule)
- "constant_with_warmup" = [`get_constant_schedule_with_warmup`] (/docs/transformers/main/en/main_classes/optimizer_schedules#transformers.optimization.schedules.ConstantScheduleWithWarmup)

<span class="code-

inline">transformers.get_schedulertransformers.get_schedulerhtt

```
transformers.get_schedulertransformers.get_schedulerhttps://github.com,
"name", "val": ":", typing.Union[str,
transformers.trainer_utils.SchedulerType ]}, {"name": "optimizer", "val": ":
Optimizer"}, {"name": "num_warmup_steps", "val": ": typing.Optional[int] =
None"}, {"name": "num_training_steps", "val": ": typing.Optional[int] = None"},
{"name": "scheduler_specific_kwargs", "val": ": typing.Optional[dict] = None"}]-
**name** ( str or SchedulerType ) --
```

The name of the scheduler to use.

- ****optimizer**** (torch.optim.Optimizer) --

The optimizer that will be used during training.

- ****num_warmup_steps**** (int , *optional*) --

The number of warmup steps to do. This is not required by all schedulers (hence the argument being

optional), the function will raise an error if it's unset and the scheduler type requires it.

- ****num_training_steps**** (int , *optional*) --

The number of training steps to do. This is not required by all schedulers (hence the argument being

optional), the function will raise an error if it's unset and the scheduler type

requires it.

<span class="code-

inline">transformers.get_constant_scheduletransformers.get_cor

```
transformers.get_constant_scheduletransformers.get_constant_scheduleht
"optimizer", "val": ": Optimizer"}, {"name": "last_epoch", "val": ": int = -1"}]-
**optimizer** (~torch.optim.Optimizer ) --
```

The optimizer for which to schedule the learning rate.

- **last_epoch** (int, *optional*, defaults to -1) --

The index of the last epoch when resuming training.0torch.optim.lr_scheduler.LambdaLR with the appropriate schedule.

Create a schedule with a constant learning rate, using the learning rate set in optimizer.

<span class="code-

inline">transformers.get_constant_schedule_with_warmuptransfc

```
transformers.get_constant_schedule_with_warmuptransformers.get_constan
"optimizer", "val": ": Optimizer"}, {"name": "num_warmup_steps", "val": ": int"},
{"name": "last_epoch", "val": ": int = -1"}]- **optimizer**
(~torch.optim.Optimizer ) --
```

The optimizer for which to schedule the learning rate.

- `**num_warmup_steps** (int) --`

The number of steps for the warmup phase.

- `**last_epoch** (int, *optional*, defaults to -1) --`

The index of the last epoch when resuming training. `torch.optim.lr_scheduler.LambdaLR` with the appropriate schedule.

Create a schedule with a constant learning rate preceded by a warmup period during which the learning rate

increases linearly between 0 and the initial lr set in the optimizer.

`<span class="code-`

`inline">transformers.get_cosine_schedule_with_warmuptransformers`

```
transformers.get_cosine_schedule_with_warmuptransformers.get_cosine_sc
"optimizer", "val": ": Optimizer"}, {"name": "num_warmup_steps", "val": ": int"},
{"name": "num_training_steps", "val": ": int"}, {"name": "num_cycles", "val": ":
float = 0.5"}, {"name": "last_epoch", "val": ": int = -1"}]- **optimizer**
(~torch.optim.Optimizer) --
```

The optimizer for which to schedule the learning rate.

- `**num_warmup_steps** (int) --`

The number of steps for the warmup phase.

- `**num_training_steps**` (int) --

The total number of training steps.

- `**num_cycles**` (float, **optional**, defaults to 0.5) --

The number of waves in the cosine `schedule` (the default is to just decrease from the max value to 0

following a half-cosine).

- `**last_epoch**` (int, **optional**, defaults to -1) --

`<span class="code-`

`inline">transformers.get_cosine_with_hard_restarts_schedule_wi`

```
transformers.get_cosine_with_hard_restarts_schedule_with_warmuptransformers.get_cosine_with_hard_restarts_schedule_with_warmup(optimizer, "val": ": Optimizer"}, {"name": "num_warmup_steps", "val": ": int"}, {"name": "num_training_steps", "val": ": int"}, {"name": "num_cycles", "val": ": int = 1"}, {"name": "last_epoch", "val": ": int = -1"}]- **optimizer** (~torch.optim.Optimizer) --
```

The optimizer for which to schedule the learning rate.

- `**num_warmup_steps**` (int) --

The number of steps for the warmup phase.

- **num_training_steps** (int) --

The total number of training steps.

- **num_cycles** (int, *optional*, defaults to 1) --

The number of hard restarts to use.

- **last_epoch** (int, *optional*, defaults to -1) --

The index of the last epoch when resuming training. `torch.optim.lr_scheduler.LambdaLR` with the appropriate schedule.

`<span class="code-`

`inline">transformers.get_cosine_with_min_lr_schedule_with_warr`

```
transformers.get_cosine_with_min_lr_schedule_with_warmuptransformers.g
"optimizer", "val": ": Optimizer"}, {"name": "num_warmup_steps", "val": ": int"},
{"name": "num_training_steps", "val": ": int"}, {"name": "num_cycles", "val": ":
float = 0.5"}, {"name": "last_epoch", "val": ": int = -1"}, {"name": "min_lr", "val": ":
typing.Optional[float] = None"}, {"name": "min_lr_rate", "val": ":
typing.Optional[float] = None"}- **optimizer** (~torch.optim.Optimizer) --
```

The optimizer for which to schedule the learning rate.

- **num_warmup_steps** (int) --

The number of steps for the warmup phase.

- `**num_training_steps** (int ) --`

The total number of training steps.

- `**num_cycles** (float , *optional*, defaults to 0.5) --`

The number of waves in the cosine `schedule` (the defaults is to just decrease from the max value to 0

following a half-cosine).

- `**last_epoch** (int , *optional*, defaults to -1) --`

`<span class="code-`

`inline">transformers.get_cosine_with_min_lr_schedule_with_warr`

```
transformers.get_cosine_with_min_lr_schedule_with_warmup_lr_rate(
    "optimizer", "val": ": Optimizer"}, {"name": "num_warmup_steps", "val": ": int"},
    {"name": "num_training_steps", "val": ": int"}, {"name": "num_cycles", "val": ":
float = 0.5"}, {"name": "last_epoch", "val": ": int = -1"}, {"name": "min_lr", "val": ":
typing.Optional[float] = None"}, {"name": "min_lr_rate", "val": ":
typing.Optional[float] = None"}, {"name": "warmup_lr_rate", "val": ":
typing.Optional[float] = None"}]- **optimizer** (~torch.optim.Optimizer ) --
```

The optimizer for which to schedule the learning rate.

- `**num_warmup_steps** (int ) --`

The number of steps for the warmup phase.

- `**num_training_steps**` (int) `--`

The total number of training steps.

- `**num_cycles**` (float, **optional**, defaults to 0.5) `--`

The number of waves in the cosine `schedule` (the defaults is to just decrease from the max value to 0

following a half-cosine).

- `**last_epoch**` (int, **optional**, defaults to -1) `--`

`<span class="code-`

`inline">transformers.get_linear_schedule_with_warmuptransform`

```
transformers.get_linear_schedule_with_warmuptransformers.get_linear_sc
"optimizer", "val": ""}, {"name": "num_warmup_steps", "val": ""}, {"name":
"num_training_steps", "val": ""}, {"name": "last_epoch", "val": " = -1"}]-
**optimizer** (~torch.optim.Optimizer) --
```

The optimizer for which to schedule the learning rate.

- `**num_warmup_steps**` (int) `--`

The number of steps for the warmup phase.

- `**num_training_steps**` (int) `--`

The total number of training steps.

- `**last_epoch**` (int, **optional**, defaults to -1) --

The index of the last epoch when resuming training.0torch.optim.lr_scheduler.LambdaLR` with the appropriate schedule.

Create a schedule with a learning rate that decreases linearly from t